

1. N-Queens Visualization

Program:

```
#include <stdio.h>

void solve(int n,int pos[],int row,int col) {
    if(row==n) {
        for(int i=0;
            i<n;
            i++)printf("%d ",pos[i]);
        printf("\n");
        return;
    }
    for(int c=0;
        c<n;
        c++) {
        int ok=1;
        for(int r=0;
            r<row;
            r++)if(pos[r]==c||pos[r]-r==c-row||pos[r]+r==c+row)ok=0;
        if(ok) {
            pos[row]=c;
            solve(n,pos,row+1,col);
            break;
        }
    }
}

void printBoard(int n,int pos[]) {
    for(int r=0;
        r<n;
        r++) {
        for(int c=0;
            c<n;
            c++)printf("%c ",pos[r]==c?'Q':'.');
        printf("\n");
    }
}

int main() {
    int p[8];
    int sol4[]={
        1,3,0,2
    }
    ,sol5[]={
        0,2,4,1,3
    }
    ,sol8[]={
        0,4,7,5,2,6,1,3
    }
```

```

}
;
printf("N = 4\n");
printBoard(4,sol4);
printf("\nN = 5\n");
printBoard(5,sol5);
printf("\nN = 8\n");
printBoard(8,sol8);
return 0;
}

```

Output:

```

Command Prompt
C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 6\q1.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 6\q1.exe
N = 4
. Q . .
. . . Q
Q . . .
. . Q .

N = 5
Q . . . .
. . Q . .
. . . . Q
. Q . . .
. . . Q .

N = 8
Q . . . . .
. . . . Q . .
. . . . . Q
. . . . . Q .
. . Q . . . .
. . . . . Q .
. Q . . . . .
. . . Q . . .

```

2. Generalized N-Queens

Program:

```
#include <stdio.h>
int safe(int p[],int row,int c) {
    for(int r=0;
        r<row;
        r++)if(p[r]==c||p[r]-r==c-row||p[r]+r==c+row)return 0;
    return 1;
}
int solve(int R,int C,int p[],int r,int restrictedCol1,int restrictedCol2,int obs[][2],int no) {
    if(r==R)return 1;
    for(int c=0;
        c<C;
        c++) {
        if(r==0 && (c==restrictedCol1 || c==restrictedCol2)) continue;
        int blocked=0;
        for(int i=0;
            i<no;
            i++)if(obs[i][0]==r+1&&obs[i][1]==c+1)blocked=1;
        if(!blocked && safe(p,r,c)) {
            p[r]=c;
            if(solve(R,C,p,r+1,restrictedCol1,restrictedCol2,obs,no))return 1;
        }
    }
    return 0;
}
void show(int R,int C,int p[],int obs[][2],int no) {
    for(int r=0;
        r<R;
        r++) {
        for(int c=0;
            c<C;
            c++) {
            int b=0;
            for(int i=0;
                i<no;
                i++)if(obs[i][0]==r+1&&obs[i][1]==c+1)b=1;
            printf("%c ",b?'X':(p[r]==c?'Q':'));
        }
        printf("\n");
    }
}
int main() {
    int p[10],o1[1][2]= {
        {
            0,0
        }
    }
}
```

```

}
;
printf("8x10 board: columns = ");
int a[]= {
    1,3,5,7,9,2,4,6
}
;
for(int i=0;
i<8;
i++)printf("%d ",a[i]);
printf("\n");
int obs2[2][2]= {
    {
        2,2
    }
    , {
        4,4
    }
}
;
if(solve(5,5,p,0,-1,-1,obs2,2)) {
    printf("5x5 with obstacles:\n");
    show(5,5,p,obs2,2);
}
int obs3[1][2]= {
    {
        0,0
    }
}
;
if(solve(6,6,p,0,1,3,obs3,0)) {
    printf("6x6 restricted positions:\n");
    show(6,6,p,obs3,0);
}
return 0;
}

```

Output:

```

Command Prompt
C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 6\q2.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 6\q2.exe
8x10 board: columns = 1 3 5 7 9 2 4 6
5x5 with obstacles:
Q . . . .
. X Q . .
. . . . Q
. Q . X .
. . . Q .
6x6 restricted positions:
. . Q . . .
. . . . . Q
. Q . . . .
. . . . Q .
Q . . . . .
. . . Q . .

```


3. Sudoku Solver

Program:

```
#include <stdio.h>
int a[9][9]= {
    {
        5,3,0,0,7,0,0,0,0
    }
    , {
        6,0,0,1,9,5,0,0,0
    }
    , {
        0,9,8,0,0,0,0,6,0
    }
    , {
        8,0,0,0,6,0,0,0,3
    }
    , {
        4,0,0,8,0,3,0,0,1
    }
    , {
        7,0,0,0,2,0,0,0,6
    }
    , {
        0,6,0,0,0,0,2,8,0
    }
    , {
        0,0,0,4,1,9,0,0,5
    }
    , {
        0,0,0,0,8,0,0,7,9
    }
}
;
int safe(int r,int c,int v) {
    for(int i=0;
        i<9;
        i++)if(a[r][i]==v||a[i][c]==v)return 0;
    int R=r/3*3,C=c/3*3;
    for(int i=R;
        i<R+3;
        i++)for(int j=C;
            j<C+3;
            j++)if(a[i][j]==v)return 0;
    return 1;
}
int solve() {
    for(int r=0;
```

```

r<9;
r++)for(int c=0;
c<9;
c++)if(!a[r][c]) {
    for(int v=1;
    v<=9;
    v++)if(safe(r,c,v)) {
        a[r][c]=v;
        if(solve())return 1;
        a[r][c]=0;
    }
    return 0;
}
return 1;
}
int main() {
    solve();
    for(int r=0;
    r<9;
    r++) {
        for(int c=0;
        c<9;
        c++)printf("%d ",a[r][c]);
        printf("\n");
    }
    return 0;
}

```

Output:

```

Command Prompt
C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 6\q3.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 6\q3.exe
5 3 4 6 7 8 9 1 2
6 7 2 1 9 5 3 4 8
1 9 8 3 4 2 5 6 7
8 5 9 7 6 1 4 2 3
4 2 6 8 5 3 7 9 1
7 1 3 9 2 4 8 5 6
9 6 1 5 3 7 2 8 4
2 8 7 4 1 9 6 3 5
3 4 5 2 8 6 1 7 9

```

4. Sudoku Solver (Backtracking)

Program:

```
#include <stdio.h>
int a[9][9]= {
    {
        5,3,0,0,7,0,0,0,0
    }
    , {
        6,0,0,1,9,5,0,0,0
    }
    , {
        0,9,8,0,0,0,0,6,0
    }
    , {
        8,0,0,0,6,0,0,0,3
    }
    , {
        4,0,0,8,0,3,0,0,1
    }
    , {
        7,0,0,0,2,0,0,0,6
    }
    , {
        0,6,0,0,0,0,2,8,0
    }
    , {
        0,0,0,4,1,9,0,0,5
    }
    , {
        0,0,0,0,8,0,0,7,9
    }
}
;
int safe(int r,int c,int v) {
    for(int i=0;
        i<9;
        i++)if(a[r][i]==v||a[i][c]==v)return 0;
    int R=r/3*3,C=c/3*3;
    for(int i=R;
        i<R+3;
        i++)for(int j=C;
            j<C+3;
            j++)if(a[i][j]==v)return 0;
    return 1;
}
int solve() {
    for(int r=0;
```

```

r<9;
r++)for(int c=0;
c<9;
c++)if(!a[r][c]) {
    for(int v=1;
    v<=9;
    v++)if(safe(r,c,v)) {
        a[r][c]=v;
        if(solve())return 1;
        a[r][c]=0;
    }
    return 0;
}
return 1;
}
int main() {
    solve();
    for(int r=0;
    r<9;
    r++) {
        for(int c=0;
        c<9;
        c++)printf("%d ",a[r][c]);
        printf("\n");
    }
    return 0;
}

```

Output:

```

Command Prompt
C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 6\q4.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 6\q4.exe
5 3 4 6 7 8 9 1 2
6 7 2 1 9 5 3 4 8
1 9 8 3 4 2 5 6 7
8 5 9 7 6 1 4 2 3
4 2 6 8 5 3 7 9 1
7 1 3 9 2 4 8 5 6
9 6 1 5 3 7 2 8 4
2 8 7 4 1 9 6 3 5
3 4 5 2 8 6 1 7 9

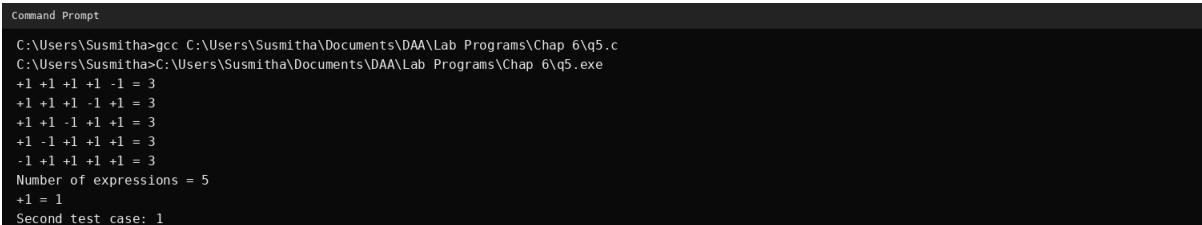
```

5. Target Sum Expressions

Program:

```
#include <stdio.h>
int nums[] = {
    1, 1, 1, 1, 1
}
,n=5,target=3,count=0,sign[10];
void dfs(int i,int sum) {
    if(i==n) {
        if(sum==target) {
            count++;
            for(int j=0;
                j<n;
                j++)printf("%c%d ",sign[j]>0?'+':'-',nums[j]);
            printf("= %d\n",sum);
        }
        return;
    }
    sign[i]=1;
    dfs(i+1,sum+nums[i]);
    sign[i]=-1;
    dfs(i+1,sum-nums[i]);
}
int main() {
    dfs(0,0);
    printf("Number of expressions = %d\n",count);
    int oldn=n;
    n=1;
    nums[0]=1;
    target=1;
    count=0;
    dfs(0,0);
    printf("Second test case: %d\n",count);
    return 0;
}
```

Output:



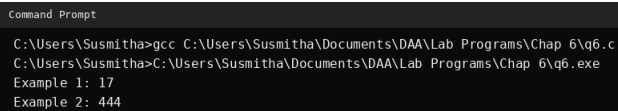
```
Command Prompt
C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap_6\q5.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap_6\q5.exe
+1 +1 +1 +1 -1 = 3
+1 +1 +1 -1 +1 = 3
+1 +1 -1 +1 +1 = 3
+1 -1 +1 +1 +1 = 3
-1 +1 +1 +1 +1 = 3
Number of expressions = 5
+1 = 1
Second test case: 1
```

6. Sum of Subarray Minimums

Program:

```
#include <stdio.h>
#define MOD 1000000007LL
long long solve(int a[],int n) {
    long long ans=0;
    for(int i=0;
        i<n;
        i++) {
        int L=1,R=1;
        for(int j=i-1;
            j>=0&& a[j]>a[i];
            j--)L++;
        for(int j=i+1;
            j<n&& a[j]>=a[i];
            j++)R++;
        ans=(ans+(long long)a[i]*L*R)%MOD;
    }
    return ans;
}
int main() {
    int a[]={
        3,1,2,4
    },b[]={
        11,81,94,43,3
    };
    printf("Example 1: %lld\n",solve(a,4));
    printf("Example 2: %lld\n",solve(b,5));
    return 0;
}
```

Output:



```
Command Prompt
C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap_6\q6.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap_6\q6.exe
Example 1: 17
Example 2: 444
```

7. Combination Sum

Program:

```
#include <stdio.h>
int c[] = {
    2,3,6,7
}
,n=4,t=7,path[20],pc=0;
void dfs(int start,int sum) {
    if(sum==t) {
        printf("");
        for(int i=0;
            i<pc;
            i++)printf("%d%s",path[i],i+1==pc?"":" ");
        printf("\n");
        return;
    }
    for(int i=start;
        i<n;
        i++)if(sum+c[i]<=t) {
        path[pc++]=c[i];
        dfs(i,sum+c[i]);
        pc--;
    }
}
int main() {
    printf("Example 1:\n");
    dfs(0,0);
    int d[] = {
        2,3,5
    }
    ;
    for(int i=0;
        i<3;
        i++)c[i]=d[i];
    n=3;
    t=8;
    printf("Example 2:\n");
    dfs(0,0);
    return 0;
}
```

Output:

Command Prompt

```
C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 6\q7.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 6\q7.exe
Example 1:
[2,2,3]
[7]
Example 2:
[2,2,2,2]
[2,3,3]
[3,5]
```

8. Combination Sum II

Program:

```
#include <stdio.h>
int c[] = {
    10, 1, 2, 7, 6, 1, 5
}
, n = 7, t = 8, path[20], pc = 0;
void sort() {
    for(int i = 0;
        i < n;
        i++)
        for(int j = i + 1;
            j < n;
            j++)
            if(c[j] < c[i]) {
                int x = c[i];
                c[i] = c[j];
                c[j] = x;
            }
}
void dfs(int start, int sum) {
    if(sum == t) {
        printf("[");
        for(int i = 0;
            i < pc;
            i++)
            printf("%d%s", path[i], i + 1 == pc ? "": ", ");
        printf("]\n");
        return;
    }
    for(int i = start;
        i < n;
        i++) {
        if(i > start && c[i] == c[i - 1]) continue;
        if(sum + c[i] > t) break;
        path[pc++] = c[i];
        dfs(i + 1, sum + c[i]);
        pc--;
    }
}
int main() {
    sort();
    printf("Example 1:\n");
    dfs(0, 0);
    int d[] = {
        2, 5, 2, 1, 2
    }
    ;
    for(int i = 0;
        i < 5;
```

```
        i++)c[i]=d[i];  
    n=5;  
    t=5;  
    sort();  
    printf("Example 2:\n");  
    dfs(0,0);  
    return 0;  
}
```

Output:

```
Command Prompt  
C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 6\q8.c  
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 6\q8.exe  
Example 1:  
[1,1,6]  
[1,2,5]  
[1,7]  
[2,6]  
Example 2:  
[1,2,2]  
[5]
```

9. Permutations

Program:

```
#include <stdio.h>
int a[] = {
    1,2,3
}
,n=3,u[3],p[3];
void dfs(int k) {
    if(k==n) {
        printf("[");
        for(int i=0;
            i<n;
            i++)printf("%d%s",p[i],i==n-1?"":",");
        printf("]\n");
        return;
    }
    for(int i=0;
        i<n;
        i++)if(!u[i]) {
        u[i]=1;
        p[k]=a[i];
        dfs(k+1);
        u[i]=0;
    }
}
int main() {
    dfs(0);
    return 0;
}
```

Output:

```
Command Prompt
C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 6\q9.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 6\q9.exe
[1,2,3]
[1,3,2]
[2,1,3]
[2,3,1]
[3,1,2]
[3,2,1]
```

10. Permutations II

Program:

```
#include <stdio.h>
int a[] = {
    1,1,2
}
,n=3,u[3],p[3];
void dfs(int k) {
    if(k==n) {
        printf("[");
        for(int i=0;
            i<n;
            i++)printf("%d%s",p[i],i==n-1?"":",");
        printf("]\n");
        return;
    }
    for(int i=0;
        i<n;
        i++)if(!u[i]&&(i==0||a[i]!=a[i-1]||u[i-1])) {
        u[i]=1;
        p[k]=a[i];
        dfs(k+1);
        u[i]=0;
    }
}
int cmp(int x,int y) {
    return x-y;
}
int main() {
    for(int i=0;
        i<n;
        i++)for(int j=i+1;
            j<n;
            j++)if(a[j]<a[i]) {
        int x=a[i];
        a[i]=a[j];
        a[j]=x;
    }
    dfs(0);
    return 0;
}
```

Output:

Command Prompt

```
C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 6\q10.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 6\q10.exe
[1,1,2]
[1,2,1]
[2,1,1]
```

11. Graph Coloring – Minimum Colors

Program:

```
#include <stdio.h>
int n=4,k=3;
int g[4][4]= {
    {
        0,1,1,1
    }
    , {
        1,0,1,0
    }
    , {
        1,1,0,1
    }
    , {
        1,0,1,0
    }
}
, col[4];
int safe(int v,int c) {
    for(int u=0;
        u<n;
        u++)if(g[v][u]&&col[u]==c)return 0;
    return 1;
}
int color(int v) {
    if(v==n)return 1;
    for(int c=1;
        c<=k;
        c++)if(safe(v,c)) {
        col[v]=c;
        if(color(v+1))return 1;
        col[v]=0;
    }
    return 0;
}
int main() {
    if(color(0)) {
        int used=0;
        for(int i=0;
            i<n;
            i++)if(col[i]>used)used=col[i];
        printf("Minimum colors needed = %d\n",used);
        printf("Color assignment: ");
        for(int i=0;
            i<n;
            i++)printf("%d ",col[i]);
    }
```

```
    printf("\n");  
}  
return 0;  
}
```

Output:



```
Command Prompt  
C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 6\q11.c  
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 6\q11.exe  
Minimum colors needed = 3  
Color assignment: 1 2 3 2
```

12. Graph Coloring with k Colors

Program:

```
#include <stdio.h>
int n=4,k=3,g[4][4]= {
    {
        0,1,1,1
    }
    , {
        1,0,1,0
    }
    , {
        1,1,0,1
    }
    , {
        1,0,1,0
    }
}
,col[4];
int safe(int v,int c) {
    for(int u=0;
        u<n;
        u++)if(g[v][u]&&col[u]==c)return 0;
    return 1;
}
int solve(int v) {
    if(v==n)return 1;
    for(int c=1;
        c<=k;
        c++)if(safe(v,c)) {
        col[v]=c;
        if(solve(v+1))return 1;
        col[v]=0;
    }
    return 0;
}
int main() {
    printf("Minimum colors for graph = 3\n");
    if(solve(0)) {
        printf("Color assignment: ");
        for(int i=0;
            i<n;
            i++)printf("%d ",col[i]);
        printf("\n");
    }
    return 0;
}
```

Output:

Command Prompt

```
C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 6\q12.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 6\q12.exe
Minimum colors for graph = 3
Color assignment: 1 2 3 2
```

13. Hamiltonian Cycle

Program:

```
#include <stdio.h>
int n=5,adj[5][5]= {
    {
        0,1,0,1,1
    }
    , {
        1,0,1,0,0
    }
    , {
        0,1,0,1,1
    }
    , {
        1,0,1,0,1
    }
    , {
        1,0,1,1,0
    }
}
,p[5],u[5];
int dfs(int pos) {
    if(pos==n)return adj[p[n-1]][p[0]];
    for(int v=1;
        v<n;
        v++)if(!u[v]&&adj[p[pos-1]][v]) {
        u[v]=1;
        p[pos]=v;
        if(dfs(pos+1))return 1;
        u[v]=0;
    }
    return 0;
}
int main() {
    p[0]=0;
    u[0]=1;
    printf("Hamiltonian cycle exists: %s\n",dfs(1)?"True":"False");
    if(adj[p[4]][p[0]]) {
        printf("Cycle: ");
        for(int i=0;
            i<n;
            i++)printf("%d ",p[i]);
        printf("0\n");
    }
    return 0;
}
```

Output:

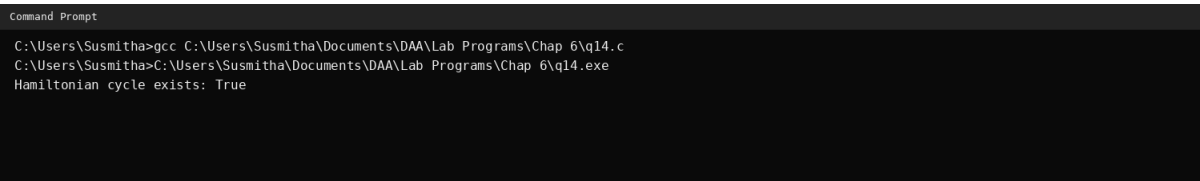
```
Command Prompt
C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 6\q13.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 6\q13.exe
Hamiltonian cycle exists: True
Cycle: 0 1 2 3 4 0
```

14. Hamiltonian Cycle (Second Case)

Program:

```
#include <stdio.h>
int n=4,adj[4][4]= {
    {
        0,1,1,1
    }
    , {
        1,0,1,0
    }
    , {
        1,1,0,1
    }
    , {
        1,0,1,0
    }
}
,p[4],u[4];
int dfs(int pos) {
    if(pos==n)return adj[p[n-1]][0];
    for(int v=1;
        v<n;
        v++)if(!u[v]&&adj[p[pos-1]][v]) {
        u[v]=1;
        p[pos]=v;
        if(dfs(pos+1))return 1;
        u[v]=0;
    }
    return 0;
}
int main() {
    p[0]=0;
    u[0]=1;
    printf("Hamiltonian cycle exists: %s\n",dfs(1)?"True":"False");
    return 0;
}
```

Output:



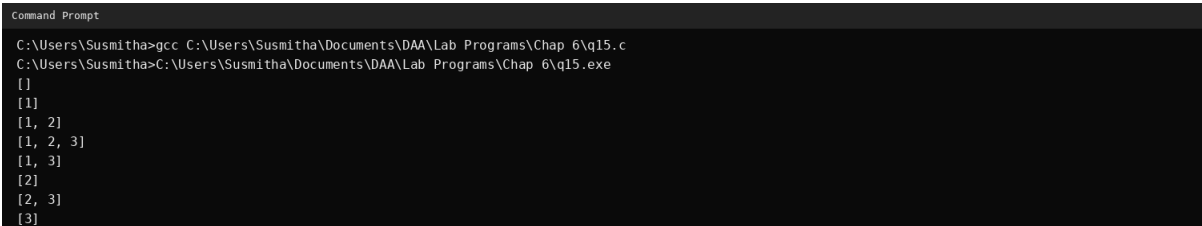
```
Command Prompt
C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 6\q14.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 6\q14.exe
Hamiltonian cycle exists: True
```

15. Subset Generation

Program:

```
#include <stdio.h>
int a[] = {
    1, 2, 3
}
,n=3,p[3],m=0;
void print() {
    printf("[");
    for(int i=0;
        i<m;
        i++)printf("%d%s",p[i],i+1==m?"":", ");
    printf("]\n");
}
void dfs(int start) {
    print();
    for(int i=start;
        i<n;
        i++) {
        p[m++] = a[i];
        dfs(i+1);
        m--;
    }
}
int main() {
    dfs(0);
    return 0;
}
```

Output:



```
Command Prompt
C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAALab Programs\Chap_6\q15.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAALab Programs\Chap_6\q15.exe
[]
[1]
[1, 2]
[1, 2, 3]
[1, 3]
[2]
[2, 3]
[3]
```

16. Subsets Containing a Given Element

Program:

```
#include <stdio.h>
int a[] = {
    2, 3, 4, 5
}
,n=4,p[4],m=0;
void print() {
    printf("[");
    for(int i=0;
        i<m;
        i++)printf("%d%s",p[i],i+1==m?"":", ");
    printf("]\n");
}
void gen(int start,int need,int has3) {
    if(m==need) {
        if(has3)print();
        return;
    }
    for(int i=start;
        i<n;
        i++) {
        p[m++]=a[i];
        gen(i+1,need,has3||(a[i]==3));
        m--;
    }
}
int main() {
    for(int size=1;
        size<=n;
        size++)gen(0,size,0);
    return 0;
}
```

Output:

```
Command Prompt
C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 6\q16.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 6\q16.exe
[3]
[2, 3]
[3, 4]
[3, 5]
[2, 3, 4]
[2, 3, 5]
[3, 4, 5]
[2, 3, 4, 5]
```

17. Word Subsets / Universal Strings

Program:

```
#include <stdio.h>
#include <string.h>
void need(const char*s,int cnt[26]) {
    for(int i=0;
        s[i];
        i++)cnt[s[i]-'a']++;
}
int universal(const char*a,const char*w2[],int m) {
    int have[26]= {
        0
    }
    ,req[26]= {
        0
    }
    ,t[26];
    need(a,have);
    for(int j=0;
        j<m;
        j++) {
        for(int z=0;
            z<26;
            z++)t[z]=0;
        need(w2[j],t);
        for(int z=0;
            z<26;
            z++)if(t[z]>req[z])req[z]=t[z];
    }
    for(int z=0;
        z<26;
        z++)if(have[z]<req[z])return 0;
    return 1;
}
int main() {
    const char*w1[]={
        "amazon","apple","facebook","google","leetcode"
    }
    ;
    const char*w2a[]={
        "e","o"
    }
    ,*w2b[]={
        "l","e"
    }
    ;
    printf("Example 1: ");
```

```
for(int i=0;
i<5;
i++)if(universal(w1[i],w2a,2))printf("%s ",w1[i]);
printf("\nExample 2: ");
for(int i=0;
i<5;
i++)if(universal(w1[i],w2b,2))printf("%s ",w1[i]);
printf("\n");
return 0;
}
```

Output:

```
Command Prompt
C:\Users\Susmitha>gcc C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 6\q17.c
C:\Users\Susmitha>C:\Users\Susmitha\Documents\DAA\Lab Programs\Chap 6\q17.exe
Example 1: facebook google leetcode
Example 2: apple google leetcode
```